

Reinforcement Learning

Reinforcement Learning

RL is it used to **learn actions** and has been successfully applied to several control tasks:

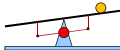
- videogames



- strategy games



- robot control

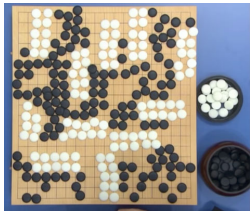


- autonomous driving



Dominating Go

- RL has been used by Google DeepMind to master the game of **Go**.
Go (# pos. $\sim 10^{170}$) is more complex than chess (# pos. $\sim 10^{120}$).
- On **May 23, 2017**, **AlphaGo Zero** defeated world's Go champion **Ke Jie**.
 - AlphaGo Zero used deep learning to learn from its own errors!



AlphaGo vs. Deep Blue

Deep Blue and AlphaGo Zero are fundamentally different:

Deep Blue used brute force to search positions using historical data and human coded heuristic functions.



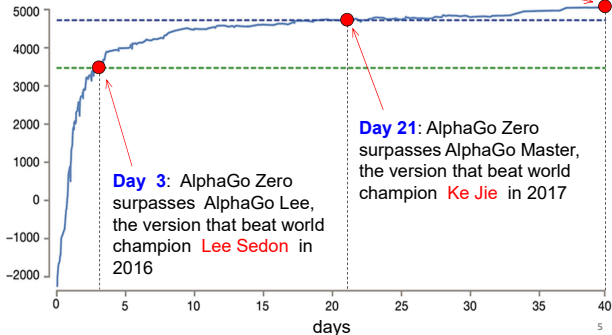
Alpha Go Zero learned from scratch by self playing with no human intervention and no historical data.



AlphaGo Zero learning curve

Elo
Rating

Day 40: AlphaGo Zero surpasses all previous versions, becoming the best Go player in the world.



Is Go important?

You may not care about Go and strategy games in general, since there are several more important areas, as robotics, medicine, intelligent transport systems, etc.



But the same neural methodology used to learn Go can be used to learn very different activities, as **walking**, **biking**, and **driving**.

Elements of RL

Key elements of RL

- It is used to **control** an **agent**;
- There is **no supervisor**, only a feedback signal (**reward**);
- Feedback is **delayed**, not instantaneous;
- The agent has to **explore** to find the right actions;
- The goal is to **maximize the cumulative future reward**.

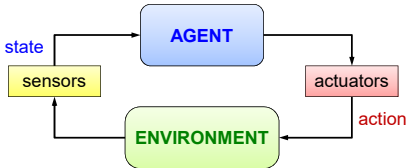
Note that, in order to explore, at the beginning of learning, actions need to be **probabilistic** and slowly become deterministic as the agent acquires more knowledge about the environment.

Reinforcement Learning

Reinforcement Learning (RL) is a learning paradigm aimed at discovering the best actions to achieve a goal.

Consider an **agent** that interacts with an environment through sensors and actuators:

- It can read sensors to **observe the state** of the environment;
- It can perform **actions** to modify the environment.

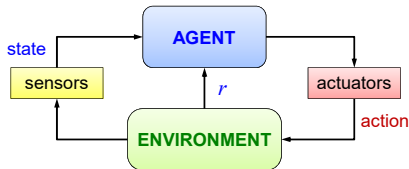


Reinforcement Learning

Occasionally, the agent also receives a feedback r from the environment (**reinforcement signal** or **reward**) that encodes how good or bad the agent is doing.

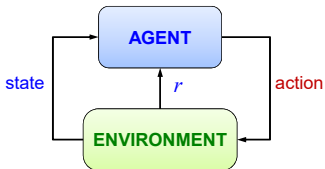
Such a reward r
is a scalar value:

$r = 0$	$\Rightarrow$ no evaluation
$r > 0$	$\Rightarrow$ good
$r < 0$	$\Rightarrow$ bad



Reinforcement Learning

From now on, we simplify the diagram, assuming that the state is acquired by proper sensors and the action is generated by proper actuators:



r is a scalar value

$r = 0$	$\Rightarrow$	no evaluation
$r > 0$	$\Rightarrow$	good
$r < 0$	$\Rightarrow$	bad

The **GOAL** of RL is to train the agent to generate actions that **maximize the total future reward**.

Reward

This means that a **goal** has to be specified in terms of **rewards**.

Examples

Task	$r > 0$	$r < 0$
Walking	go ahead	fall down / go back
Driving	follow the road	crash
Pong	hit the ball	miss the ball
Shooting	hit the target	miss the target
Playing music	note on scale	note out of scale

Rewards & Punishments

RL is similar to learning by **rewards** and **punishments**:

- A **negative reward** is equivalent to **punish** the agent;
- A **positive reward** is equivalent to **reward** the agent.



Reinforcement Learning

In contrast to supervised learning, **RL** allows an agent to learn by itself:

- Actions leading to positive rewards must be **encouraged**;
- Actions leading to negative rewards must be **discouraged**.

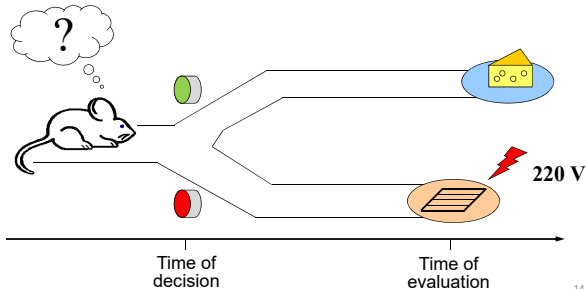
agent



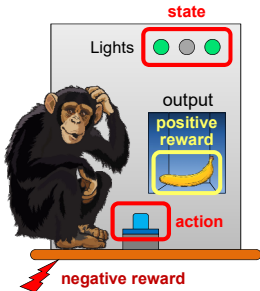
At the beginning, the agent has to **explore** by trying random actions.

Temporal credit assignment

Often, rewards are **delayed** (i.e., arrive **after many actions**).
In this cases, understanding which actions are responsible for a success or a failure is not easy.



Example

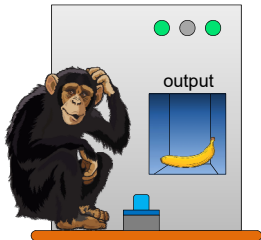


A monkey has to learn which action has to take in order to get a banana.

Depending on the state of the lights, the same action may result in

- A positive reward 
- Nothing
- A negative reward 

Example



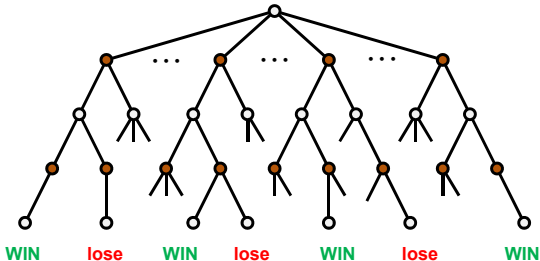
trial	state	action	reward
1	  		
2	  		
3	  		
4	  		
5	  		

Did you figure out the rule?

Rule: Button pressed when exactly 2 lights are on $\Rightarrow$ banana
 Button pressed when ≥ 2 lights are off $\Rightarrow$ electric shock

Learning in Chess

Learning Chess in this way means understanding good or bad actions only at the end of the game:

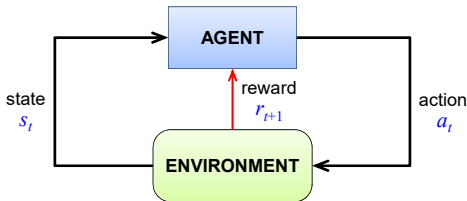


Formalizing Reinforcement Learning

The agent

An **agent** interacts with the **environment** (**world**) through **actions**.

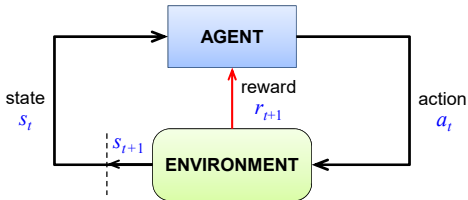
- At each time step t , the agent observes the state s_t of the environment and selects an action a_t .
- At time $t + 1$, the agent receives a **reward** r_{t+1} (a numerical value) indicating the appropriateness of action a_t in state s_t .



The environment

The **environment** evolves from a state to another depending on the **actions** taken by the **agent**.

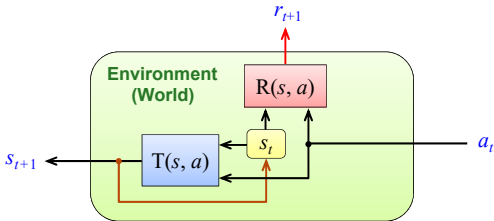
- At each time step t , the environment receives an action a_t from the agent.
- At time $t + 1$, it goes to state s_{t+1} and generates the **reward** r_{t+1} .



The environment

Internally, the **environment** uses two functions to produce the next state s_{t+1} and the reward r_{t+1} from the current state s_t and action a_t :

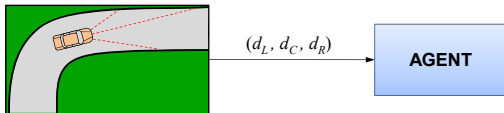
$$\begin{cases} s_{t+1} = T(s_t, a_t) & \text{State transition function} \\ r_{t+1} = R(s_t, a_t) & \text{Reward function} \end{cases}$$



Meaning of state

State Set of variables describing the environment.

Observation Set of variables available to the agent (subset of state)



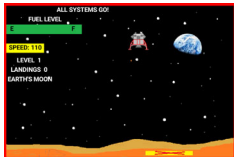
State: car position, velocity, acceleration, LiDAR distances
 $(x, y, \theta, v_x, v_y, a_x, a_y, d_L, d_C, d_R)$

Observation: distances detected by the three LiDARs (d_L, d_C, d_R)

Note: In general, the **state variables** given to the agent are less than those needed to simulate the world.

Meaning of state

In simple settings, as **Chess**, observation and state are the same, since the positions of the pieces contain the **full information** about the environment.



In other cases (images), data may be **noisy**, may include objects **irrelevant** for the agent, or may be **incomplete** (e.g., a single frame does not encode speed).

In this cases, the agent should acquire more frames or include an RNN to learn a state representation from a sequence of frames.

For simplicity, in the following, the information available to the agent is denoted just as **state**.

Important remark

In general, the response of a system could depend on everything that has happened earlier, that is:

$$(s_{t+1}, r_{t+1}) \leftarrow s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_{t-1}, a_{t-1}, r_t, s_t, a_t$$

In RL, both **state** and **reward** a time $t+1$ must depend only on the state and action at time t (**Markovian property**), that is:

$$(s_{t+1}, r_{t+1}) \leftarrow s_t, a_t$$

An environment with the **Markovian property** enables the agent to predict the **next state** and the **next reward** given the current state and action, as it would be possible to do from the complete history.

Important remark

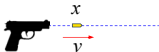
Examples of Markovian state representations:

Chess



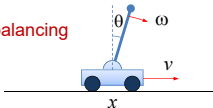
the next board position only depends on the current position and the move taken by the player.

Bullet



its position and velocity are sufficient to predict its future trajectory.

Pole balancing

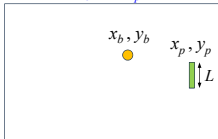


Position and velocity of the cart and angle and angular velocity of the pole are sufficient to predict its behavior.

Important remark

Non-Markovian reward: the environment computes rewards based on variables that are NOT part of the current state.

Assume v_b and v_p constant



non Markovian

Markovian

State

x_b, y_b

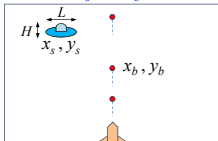
x_b, y_b, x_p, y_p

Reward

$|y_b - y_p| < L/2$
 $|x_b - x_p| < r$

$|y_b - y_p| < L/2$
 $|x_b - x_p| < r$

Assume v_s and v_b constant



non Markovian

Markovian

State

x_s, y_s

x_s, y_s, x_b, y_b

Reward

$|x_b - x_s| < L/2$
 $|y_b - y_s| < H/2$

$|x_b - x_s| < L/2$
 $|y_b - y_s| < H/2$

Important remark

Why should the Reward function be Markovian?

1. Convergence

Convergence is guaranteed assuming that the reward is Markovian. If this assumption is violated, the system may not converge or converge to a suboptimal solution.

2. Simplified learning

A Markovian reward function ensures that the RL algorithm does not need to keep track of past states or actions. Non-Markovian rewards complicate the policy optimization process and force the agent to maintain a history of states or actions to make optimal decisions, thus increasing computational complexity.

Example

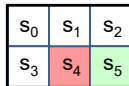
Consider an environment with 6 rooms:

Rewards:

each move: -1
 hit the wall: -2
 bad state (s_4): -10
 goal state (s_5): +10

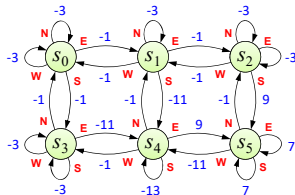
Actions:

N - North
E - East
S - South
W - West



bad
state

goal
state



Transition table

$T(s, a)$

	N	E	S	W
0	0	1	3	0
1	1	2	4	0
2	2	2	5	1
3	0	4	3	3
4	1	5	4	3
5	2	5	5	4

Reward table

$R(s, a)$

	N	E	S	W
0	-3	-1	-1	-3
1	-3	-1	-11	-1
2	-3	-3	9	-1
3	-1	-11	-3	-3
4	-1	9	-13	-1
5	-1	7	7	-11

Reward function

Note that both the **reward** and the **new state** can be computed in algorithmic form.

Rewards:

each move: -1
hit the wall: -2
bad state (s_4): -10
goal state (s_5): +10

```
int  get_reward(int s, int a)
{
  int  s_new, r;

  r = -1;
  s_new = next_state(s, a);

  if (s_new == s) r = r - 2;
  if (s_new == 4) r = r - 10;
  if (s_new == 5) r = r + 10;

  return r;
}
```

Transition function

```
int next_state(int s, int a)
{
    switch (a) {
        case NORTH: if (s < 3) return s;
                     return s - 3;

        case EAST:  if (s%3 == 2) return s;
                     return s + 1;

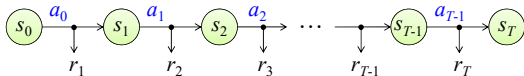
        case SOUTH: if (s >= 3) return s;
                     return s + 3;

        case WEST:  if (s%3 == 0) return s;
                     return s - 1;

        default:    printf("WRONG ACTION\n");
                     exit(1);
    }
}
```

Episode

A sequence of states, actions, and rewards from an initial state s_0 to a final state s_T is called an **episode** (or **game**):



$s_0, a_0, r_1, s_1, a_1, r_2, s_2, a_2, r_3 \dots s_{T-1}, a_{T-1}, r_T, s_T$

$T(s, a)$ is the state transition function that brings the environment into a new state: $s_{t+1} = T(s_t, a_t)$

Note that different episodes can have different length.

Usually, the **final state** s_T is defined to be

- a **goal state** (characterized by the highest reward) or
- a **failure state** (characterized by the lowest reward).

Total Reward

In one episode, the **total reward** (or **return**) is defined as:

$$R = r_1 + r_2 + \dots + r_T$$

At step t , the **total future reward** (or **future return**) is defined as:

$$R_t = r_{t+1} + r_{t+2} + \dots + r_T$$

Note that: $R_t = r_{t+1} + R_{t+1}$

GOAL of RL: in any state s , make the action a^* that maximizes the **total future reward** R_t

Maximizing the **total reward** means not only reaching the goal state but reaching it in the best possible way.

Example

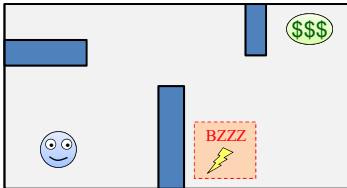
Maximizing the **total reward** means not only reaching the goal state but reaching it in the best possible way.

State: (x, y)

Actions: $\text{move}(\Delta s, \Delta \theta)$

Rewards:

- move: -1
 - obstacle: -10
 - shock: -100
 - \$\$\$: +100
- } end of episode



Example

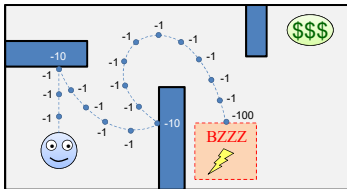
Maximizing the **total reward** means not only reaching the goal state but reaching it in the best possible way.

State: (x, y)

Actions: $\text{move}(\Delta s, \Delta \theta)$

Rewards:

- move: -1
 - obstacle: -10
 - shock: -100
 - \$\$\$: +100
- } end of episode



$$R = \underbrace{-1 -1 -1 -10}_{3 \text{ steps}} \underbrace{-1 -1 \dots -1 -10}_{5 \text{ steps}} \underbrace{-1 -1 -1 \dots -1 -1 -1 -1 -100}_{10 \text{ steps}} = -138$$

Example

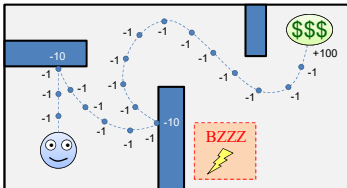
Maximizing the **total reward** means not only reaching the goal state but reaching it in the best possible way.

State: (x, y)

Actions: $\text{move}(\Delta s, \Delta \theta)$

Rewards:

- move: -1
 - obstacle: -10
 - shock: -100
 - \$\$\$: +100
- } end of episode



$$R = \underbrace{-1 -1 -1 -10}_{3 \text{ steps}} \underbrace{-1 -1 \dots -1 -10}_{5 \text{ steps}} \underbrace{-1 -1 -1 \dots -1 -1 -1 -1 -1}_{12 \text{ steps}} + 100 = +60$$

Example

Maximizing the **total reward** means not only reaching the goal state but reaching it in the best possible way.

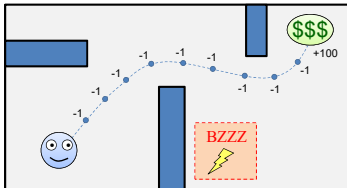
State: (x, y)

Actions: $\text{move}(\Delta s, \Delta \theta)$

Rewards:

- move: -1
- obstacle: -10
- shock: -100
- \$\$\$: +100

} end of episode

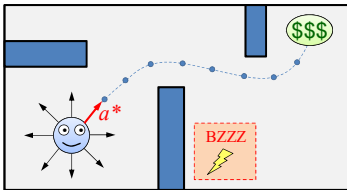


$$R = \underbrace{-1 -1 -1 -1 -1 -1 -1 -1 -1 -1}_{\text{10 steps}} + 100 = +90$$

Goal of RL

When the agent is in a state s_t , the best action a^* is the one that maximizes the **total future reward** R_t :

Hence, the agent must learn to predict in each state s_t the maximum **total future reward** R_t for reaching the goal.



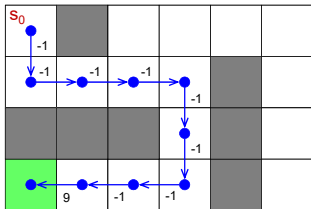
Assume for a moment we are able to predict the value of the **maximum** R_t in each state s_t . Let $V(s_t)$ be such a value.

Example

Consider this example, where the **goal** is the **green room**.
 The **reward** is **-1** for each move and **10** in the goal state.

Actions

N	north
E	east
S	south
W	west



The **total reward** accumulated for the shortest trajectory from s_0 is:

$$R = (-1) + (-1) + (-1) + (-1) + (-1) + (-1) + (-1) + (-1) + (-1) + (-1 + 10) = 1$$

Hence, the value of state s_0 is $V(s_0) = 1$.

Example of $V(s)$

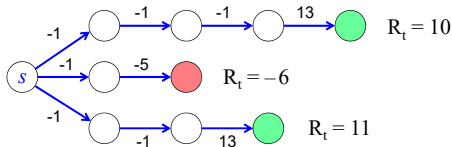
Similarly, in **each state** s , the **value** $V(s)$ that correctly predicts the maximum total reward to reach the goal from that state is given by:

1		3	4	3	2
2	3	4	5		1
			6		0
10	9	8	7		-1

Once $V(s)$ is known, it can be used by the agent to select the action that brings it to the next state with the highest value.

$V(s)$ interpretation

If the policy is **stochastic**, $V(s)$ will converge to the **average return** that can be achieved by taking all possible trajectories from s up to an end state.



In this case, after a sufficiently large number of episodes executed with random actions (with equal probability) $V(s)$ will converge to the value of $(10 - 6 + 11)/3 = 5$.

$V(s)$ interpretation

If actions have different probabilities to be selected and $\pi(a|s)$ is the probability that action a is selected from a given state s , then $V(s)$ will converge to the **weighted average** of the various returns, where the weight is the probability $\pi(a|s)$:

$$V(s) = \sum_{a \in \mathcal{A}(s)} \pi(a|s) R(s, a)$$

$\mathcal{A}(s)$ = set of possible actions from state s

$R(s, a)$ = return from state s after action a

Now, the expectation of a random variable X is defined as

$$E[X] = \sum_i p_i x_i$$

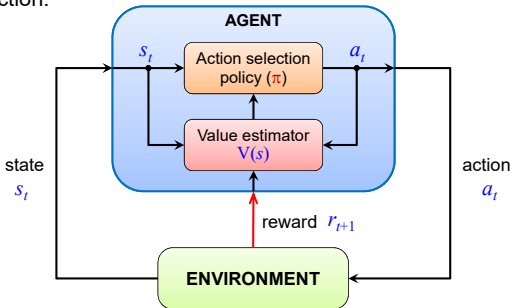
where x_i is a generic outcome of X and p_i is its probability.

Hence, $V(s)$ can be expressed as the expectation of the return R_t accumulated from the initial state s following the policy π :

$$V(s) = E_{\pi}[R_t | s_t = s]$$

Inside the agent

The agent has a **policy** $\pi(s)$ to select actions from a state s , and a **quality value estimator** for $V(s)$, used by the policy to choose an action:



Greedy policy

The **policy** is the rule that maps states to actions:

$$a = \pi(s)$$

If the agent knew the value $V(s)$ in each state, it would always select the action that brings it to the state with the highest value.

This policy is called the **greedy policy** and the action selected in this way is called the **greedy action**:

**Greedy
policy**

Select the **action** that maximizes $V(s_{t+1})$:

$$a^* = \arg \max_a V(T(s, a))$$

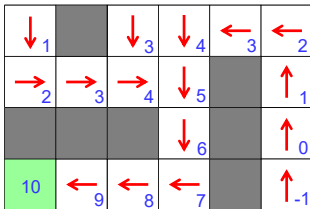
Example of greedy policy

In the previous example, the greedy policy $\pi_{gre}(s)$ that brings the agent to the goal from any state s is:

S ₁		S ₃	S ₄	W ₃	W ₂	$a^* = \pi_{gre}(s)$
E ₂	E ₃	E ₄	S ₅		N ₁	$V(s)$
			S ₆		N ₀	
10	W ₉	W ₈	W ₇		N ₋₁	

Example of greedy policy

In this case, a better way to visualize the policy is:



Greedy policy

Greedy policy

Select the **action** that maximizes $V(s_{t+1})$:

$$a^* = \arg \max_a V(T(s, a))$$

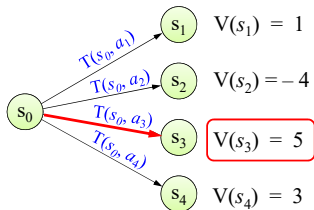
The **action** that maximizes $V(s_{t+1})$ can be implemented as follows:

```
greedy_action(s) {
    ba = a0;                                // initialize best action
    max = V(T(s, a0));                      // initialize max value
    for (i=1 to nactions-1) {
        snew = T(s, ai);                    // find the next state
        if (V(snew) > max) {
            max = V(snew);                  // update max value
            ba = ai;                        // update best action
        }
    }
    return ba;
}
```

Exploiting values

To exploit the knowledge encoded in the value function $V(s)$, the agent must know how the environment responds to actions.

If the agent knows the state transition function $T(s, a)$, it can **simulate** the consequence of each action to see which one brings to the state with the highest value:

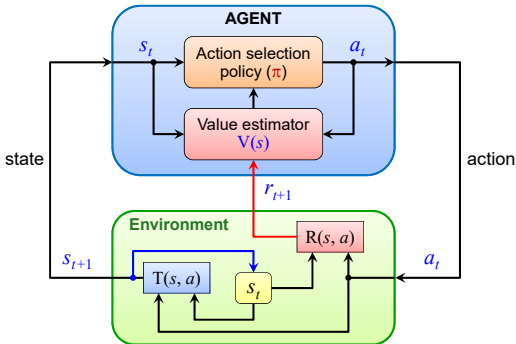


In this example, the agent can check (in simulation) all the transitions from state s_0 and discover that action a_3 is the one that leads to the highest future reward.

Then, it can actually perform action a_3 in the environment.

Exploiting values

Often, the agent has no access to the transition function $T(s, a)$:



World model

If the agent does not know the behavior of the world, it could learn a **model of the world**, by estimating the two functions

$$\left\{ \begin{array}{ll} M_T(s_t, a_t) \approx T(s_t, a_t) & \text{State transition function} \\ M_R(s_t, a_t) \approx R(s_t, a_t) & \text{Reward function} \end{array} \right.$$

Depending on whether the agent uses a knowledge about the world, RL methods are distinguished in two categories:

- Model-free methods:** the agent does not have any knowledge about the world.
- Model-based methods:** the agent has a model of the world and uses it to select better actions. The model can be **given** by the user, or **learned** from experience.

Exploitation vs. exploration

At the beginning, however, **values are unknown**, hence need to be estimated by **exploration** (i.e., trying random actions).

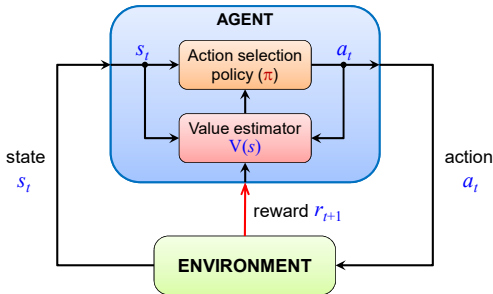
- By selecting a **non-greedy action**, the agent is **exploring**, because this enables him to find better actions and improve his estimates.
- By selecting a **greedy action**, the agent is **exploiting** his current knowledge encoded in the values.

Since it is not possible to both **exploit** and **explore** with a single action selection, exploration and exploitation are in conflict. This is known as the **exploration-exploitation dilemma**.

Balancing between exploration and exploitation is a distinctive challenge in reinforcement learning.

Action selection policy

The agent must discover which actions yield the most reward, hence actions need to be probabilistic. In a stochastic policy, $\pi(a|s)$ denotes the probability that action a is selected from a given state s .



Epsilon-greedy policy

To balance between **exploration** and **exploitation**, actions can be selected in a stochastic fashion using a probability ϵ , called **exploration factor**. This method is known as **ϵ -greedy policy**.

ϵ -greedy policy

$$a = \begin{cases} \text{random action} & \text{with probability } \epsilon \\ \text{greedy action} & \text{with probability } 1 - \epsilon \end{cases}$$

The **ϵ -greedy policy** can be implemented as follows:

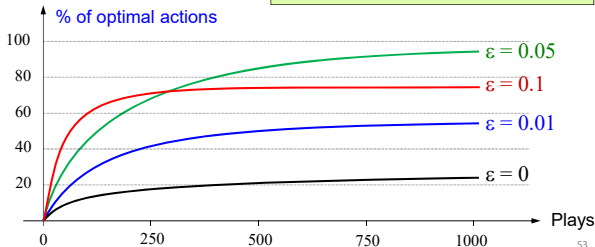
```

epsilon_greedy_policy(s, eps)
{
    x = random number in [0,1];
    if (x ≤ eps) return random_action(s);
    else       return greedy_action(s);
}
  
```

Greedy vs. ϵ -greedy

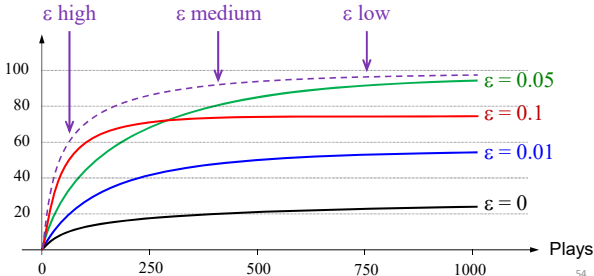
Running simulations with different values of the exploration factor ϵ confirms that a small amount of exploration is beneficial for discovering better actions.

Note that higher values of ϵ make the agent learn **faster**, but **less effectively**.



Greedy vs. ϵ -greedy

This suggests using larger values of ϵ at the beginning of learning to allow higher **exploration**, decreasing ϵ towards zero as the agent learns to allow more **exploitation**.

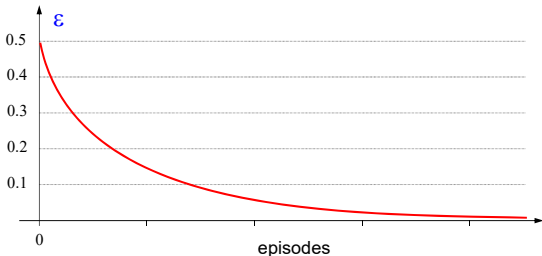


Variable ϵ

An exponential decay is typically used to decrease ϵ :

$$\epsilon = \delta \epsilon_{\text{old}}$$

where δ is the *exploration decay factor* (e.g., $\delta = 0.99$).



Boltzmann exploration

A more effective way to perform exploration is to select actions with a probability given by the **Boltzmann distribution** (also known as the **Gibbs distribution** or **softmax**) over the learned values, regulated by a **temperature parameter** τ .

If there are n actions $(a_1, \dots, a_n)$ that can be taken from state s , each bringing to a new state with value $V(s_k)$, then a_i can be selected with a probability $p_i(s)$ given by

$$p_i(s) = P(a = a_i | s) = \frac{\exp(V(s_i)/\tau)}{\sum_{k=1}^n \exp(V(s_k)/\tau)}$$

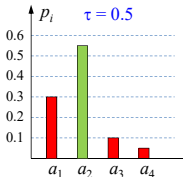
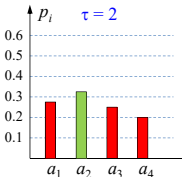
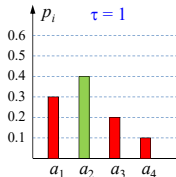
Note that, if $\tau \rightarrow \infty$, we have $p_i(s) = 1/n$, hence actions are selected randomly, while for $\tau \rightarrow 0$, the action with the highest value will be selected.

Boltzmann exploration

For example, assume $\tau = 1$ and suppose we have 4 actions:

	$V(s_1)$	$V(s_2)$	$V(s_3)$	$V(s_4)$		p_1	p_2	p_3	p_4
s	3.1	3.4	2.7	2.0	$\xrightarrow{\text{softmax}}$	0.3	0.4	0.2	0.1

Then, a_1 is selected with probability 0.3, a_2 with probability 0.4, etc.



Boltzmann exploration

To avoid overflow errors due to large exponents, the softmax is computed by subtracting the maximum value among the $V(s_k)$ and using $\beta = 1/\tau$, where β is referred to as **coldness**.

$$\left\{ \begin{array}{l} m = \max_{a_k} V(T(s, a_k)) = \max_{a_k} V(s_k) \\ p_i(s) = P(a = a_i | s) = \frac{\exp(\beta(V(s_i) - m))}{\sum_{k=1}^n \exp(\beta(V(s_k) - m))} \end{array} \right.$$

In this case, setting $\beta = 0$ corresponds to selecting a random action, whereas $\beta > 1$ makes action selection more deterministic.

Boltzmann exploration

A better implementation of the softmax uses the following trick:

$$\begin{cases} m = \max_{a_k} V(s_k) \\ sum = \sum_{k=1}^n \exp(\beta(V(s_k) - m)) \end{cases}$$

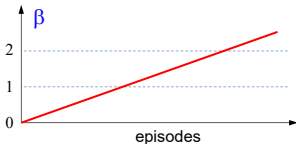
$$p_i(s) = \frac{\exp(\beta(V(s_i) - m))}{sum}$$

$$p_i(s) = \exp \left[\ln \left[\frac{\exp(\beta(V(s_i) - m))}{sum} \right] \right]$$

$$p_i(s) = \exp [\beta(V(s_i) - m) - \ln(sum)]$$

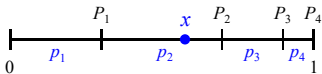
Boltzmann exploration

The **coldness** parameter β is usually increased linearly during exploration:



Actions can be selected with the following algorithm:

p_1	p_2	p_3	p_4
0.3	0.4	0.2	0.1



```

select_action(p, n) {
    x = random number in [0,1]
    P = 0;
    for (i=1 to n-1) {
        P = P + p[i];
        if (x <= P) return i;
    }
    return n;
}
    
```